

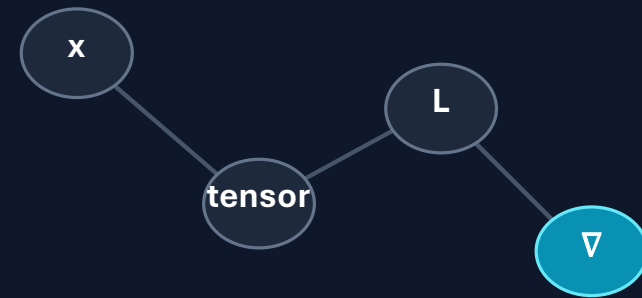
LECTURE 25

Tensors and Automatic Differentiation

From DSP arrays to computational graphs

75 minutes • tensors → shapes → broadcasting → autograd → learnable FIR

Senior + first-year graduate ECE



ROADMAP

What changes when a DSP array becomes a tensor?

The data may be familiar; the container now knows shape, device, and gradient history.

20 min

FROM VECTORS TO TENSORS

waveforms • spectra • spectrograms

one object, richer metadata

20 min

TENSOR OPERATIONS FOR DSP

matmul • broadcasting • reshape

think in shapes, not loops

35 min

CHAIN RULE + AUTOGRAD

computational graph • backward pass

learn parameters from a loss

Reading bridge

Oppenheim & Schaffer: discrete-time signals, linear transforms, convolution/STFT context • Raschka: PyTorch tensor and autograd foundations

Same signal, richer container

Start with objects students already know.

DSP notation

Finite waveform

$x[0], x[1], \dots, x[L-1]$

A sequence of samples

NumPy view

ndarray

shape = (L,)

dtype = float32

Fast vectorized CPU math

PyTorch view

Tensor

shape = (L,)

dtype = torch.float32

device = cpu / accelerator

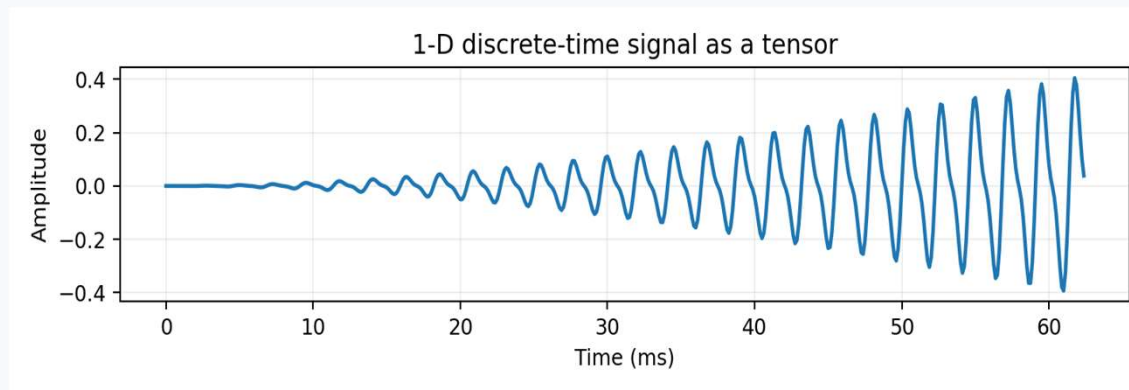
requires_grad = False / True

Key idea

A tensor is not inherently a neural-network object. It can represent the same waveform, spectrum, filterbank, or feature matrix used throughout DSP.

A 1-D discrete-time signal is already tensor-shaped

The first dimension can simply be sample index.



PyTorch representation

A finite observation with L samples is naturally stored as shape $(L,)$. Indexing $x[n]$ remains the same conceptual operation as in DSP.

Waveform tensor

```
x = torch.tensor(samples,  
dtype=torch.float32)  
print(x.shape)    # torch.Size([L])  
print(x.device)   # cpu
```

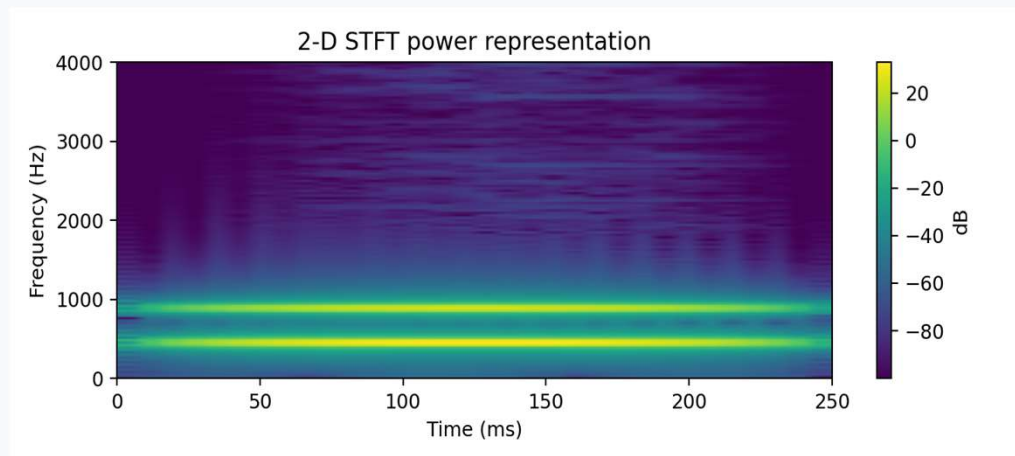
Ask:

What extra information does the tensor carry that the mathematical sequence $x[n]$ does not?

Good answer: shape, dtype, device placement, and whether operations should be tracked for gradients.

A spectrogram is naturally a rank-2 tensor

Frequency and frame index become two tensor dimensions.



$$X[k, m] = \sum_n x[n] w[n - mR] e^{-j2\pi kn/N_{\text{FFT}}}$$

Shape example

With `n_fft = 256`, a one-sided real-input STFT has 129 frequency bins. A result might have shape (129, 32): frequency × frame.

Complex dtype is okay

PyTorch tensors can be complex. Magnitude, power, phase, and differentiable STFT operations can all remain inside a computational graph.

Four properties to inspect before debugging the math

Many PyTorch errors are shape/device/dtype errors, not “ML errors.”

shape

How many axes? What does each axis mean?

(N, C, L)

dtype

float32? float64? complex64? integers?

`torch.float32`

device

Where is the memory and computation?

`cpu` / `cuda` / `mps`

requires_grad

Should PyTorch record operations for derivatives?

`True` / `False`

ECE habit

Print or assert shapes at subsystem boundaries. Treat tensor dimensions the way you treat units and signal conventions in DSP.

Similar syntax; different responsibilities

The distinction is less “array versus tensor” than “numerics versus differentiable compute.”

Question	NumPy ndarray	PyTorch tensor
Vectorized arithmetic	Yes	Yes
CPU numerical work	Excellent	Excellent
Accelerator placement	Not its core model	Built into tensor/device model
Gradient tracking	No	Yes, when <code>requires_grad=True</code>
NN layers + optimizers	External libraries	Integrated ecosystem
Interop	—	<code>torch.from_numpy</code> / <code>.numpy()</code> on CPU

Subtle example: `torch.from_numpy(a)` can share CPU memory with a NumPy array; `torch.tensor(a)` creates a new tensor copy.

GPU acceleration is a placement decision, not magic

Moving data has a cost; computation must be large enough to justify it.

CPU

general-purpose cores
low transfer overhead
great for small problems

`x = x.to(device)`

ACCELERATOR

many parallel operations
large batched tensor math
transfer + memory constraints

Important

A tensor and the model parameters that operate on it must live on compatible devices. “Expected all tensors to be on the same device” is a shape/unit-style bookkeeping error.

Map familiar audio objects to tensor shapes

Do this before introducing any new syntax.

A One mono clip, 16,000 samples

?

B 32 stereo clips, 16,000 samples each

?

C One STFT with 257 bins $\times$ 101 frames

?

D 32 one-channel spectrograms, 257×101

?

Good answers

A: (16000,) • B: (32, 2, 16000) • C: (257, 101) • D: often (32, 1, 257, 101)

Shape manipulation is part of the algorithm

The values may be correct while the axis order is wrong.

Shape operations

```
x = torch.randn(8, 4000)      # (N,L)
x = x.unsqueeze(1)           # (N,1,L)

S = torch.randn(8, 257, 101)  # (N,F,T)
S = S.unsqueeze(1)           # (N,1,F,T)

A = torch.randn(8, 20, 30)
B = A.permute(0, 2, 1)        # (8,30,20)
```

unsqueeze

Insert a length-one axis. This is how a mono batch (N,L) becomes the Conv1d convention (N,1,L).

permute

Reorder existing axes. The data are re-indexed logically; the semantic meaning of each dimension changes.

reshape / view

Change how contiguous elements are grouped. Never reshape blindly to “make the error go away”; preserve the intended axis semantics.

The common Conv1d convention: batch, channel, length

This convention matters later when CNN layers arrive.

(N, C, L)

N

Batch

independent clips

C

Channel

mono, stereo, sensor channels

L

Length

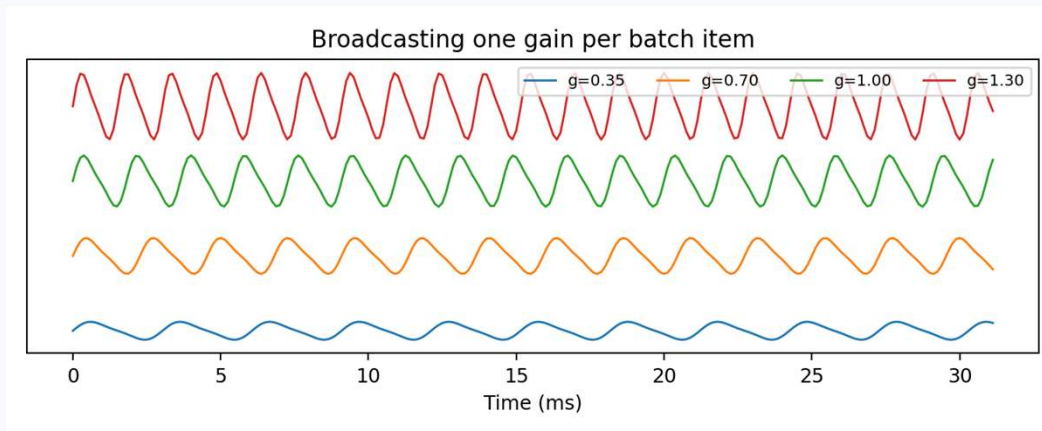
time samples

Example

32 stereo one-second clips at 16 kHz → shape (32, 2, 16000). Index $x[7, 1, 250]$ means batch item 7, channel 1, time sample 250.

Write the shape of the parameter you mean

Broadcasting replaces many explicit loops — when the dimensions line up semantically.



Good debugging question: “Which physical meaning does each axis carry?”

One gain per channel

```
audio.shape           # (N,C,L)
channel_gain = torch.tensor([0.5, 1.25])
channel_gain = channel_gain.view(1, C, 1)
y = audio * channel_gain
```

Why (1,C,1)?

Dimension 0 broadcasts across the batch. Dimension 1 matches channel. Dimension 2 broadcasts across time.

A filterbank is already a tensor operation

Classical DSP linear transforms fit naturally into PyTorch.

$$\mathbf{E} = \mathbf{W}\mathbf{P}, \quad \mathbf{W} \in \mathbb{R}^{M \times F}, \quad \mathbf{P} \in \mathbb{R}^{F \times T}$$



DSP interpretation

Each column is one time frame. The filterbank combines 257 frequency-bin powers into 40 band energies for that frame.

PyTorch

```
E = W @ P    # (40,257) @ (257,100) -> (40,100)
```

A shape error is often a model-design clue

Predict first; run second.

Given:

Will this broadcast as intended?

```
audio.shape = (8, 2, 1000)
gain.shape   = (2,)
y = audio * gain
```

Question 1

Will the operation succeed? If not, why does the last dimension cause trouble?

Question 2

What shape should gain have to mean “one gain per channel”?

Good answer

Use `gain.view(1, 2, 1)`. The semantic channel axis is the middle axis, not the final time axis.

What if the DSP coefficients are unknown?

Automatic differentiation turns a fixed signal-processing computation into a learnable one.



$$\nabla_{\theta} L = \left[\frac{\partial L}{\partial \theta_1}, \dots, \frac{\partial L}{\partial \theta_p} \right]^T$$

Question: How should each coefficient change to reduce the loss?

Autograd answers this by chaining local derivatives backward through the forward computation.

Do one derivative by hand before trusting autograd

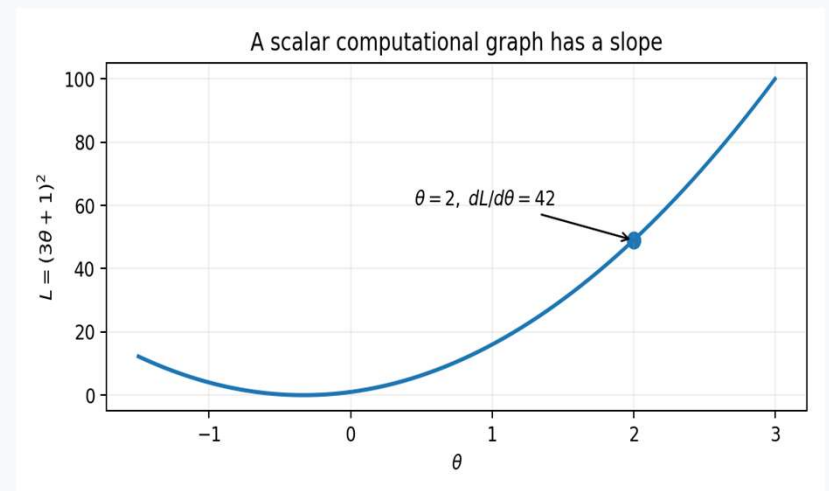
Students should see exactly what the graph will automate.

$$u = 3\theta + 1, \quad L = u^2$$

$$\frac{dL}{d\theta} = \frac{dL}{du} \frac{du}{d\theta} = 2u \cdot 3 = 6(3\theta + 1)$$

At $\theta = 2$

$u = 7$, $L = 49$, and $dL/d\theta = 42$.



Interpretation: the gradient is the local slope of the loss with respect to the parameter.

The forward pass builds a directed acyclic graph

Nodes are intermediate values; edges correspond to differentiable operations.



Reverse pass

Start from $dL/dL = 1$ and multiply local derivatives while traversing the graph backward. Shared subexpressions contribute through the chain rule.

Why backpropagation fits modern ML

One scalar loss may depend on millions of parameters.

FORWARD

Compute intermediate tensors and the scalar loss L .

Record enough local information to differentiate the operations later.



BACKWARD

Seed the output with $dL/dL = 1$.

Propagate sensitivities backward to obtain gradients for every upstream parameter that requires them.

Why efficient?

Reverse mode is well matched to “many inputs/parameters $\rightarrow$ one scalar objective.” A single backward traversal provides all components of $\nabla \theta L$.

A scalar loss can depend on a whole parameter vector

Autograd handles vectorized expressions without expanding every partial derivative by hand.

$$\hat{y}_i = wx_i + b, \quad L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Two learnable parameters

```
x = torch.linspace(-2, 2, 41)
y = 2.5*x - 0.4

w = torch.tensor(0.2, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

y_hat = w*x + b
loss = ((y_hat-y)**2).mean()
loss.backward()
```

One backward pass

PyTorch computes both $\partial L / \partial w$ and $\partial L / \partial b$ from the same graph.

Important

The forward code is still ordinary vectorized math. Gradient tracking is layered on top of it.

ECE connection

Any differentiable signal model with unknown coefficients can be treated the same way.

Four lines students need to understand

Do not let the optimizer hide the gradient mechanics.

Minimal example

```
theta = torch.tensor(2.0, requires_grad=True) # leaf parameter
u = 3*theta + 1                               # graph grows
loss = u**2                                   # scalar objective
loss.backward()                               # reverse traversal
print(theta.grad)                             # tensor(42.)
```

requires_grad=True

Track operations that influence this tensor.

loss.backward()

Run reverse-mode autodiff from the scalar loss.

.grad

Gradient accumulated on leaf tensors.

Good question: Why does theta.grad live on θ , not on every temporary tensor by default?

Answer: leaf parameters are the quantities we intend to update; retaining every intermediate gradient would waste memory unless explicitly requested.

PyTorch adds gradients; it does not replace them

This behavior is useful for deliberate accumulation and dangerous when accidental.

Accumulation demo

```
x = torch.tensor(3.0, requires_grad=True)

(x**2).backward()
print(x.grad)    # 6

(x**2).backward()
print(x.grad)    # 12 <- accumulated

x.grad.zero_()
print(x.grad)    # 0
```

Why?

A parameter can influence a loss through multiple paths. Autograd must sum contributions to the same derivative.

Normal training-loop pattern

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

`no\_grad()` and `detach()` intentionally break gradient flow

Use them because you mean to — not as a patch for a memory or type error.

`torch.no_grad()`

Context: operations performed inside it do not create autograd history. Good for evaluation, plotting predictions, or generating a fixed target.

`.detach()`

Tensor operation: return the same numerical values disconnected from the current graph. Good for inspection/export when upstream derivatives are no longer desired.

Common uses

```
with torch.no_grad():  
    y_ref = fixed_system(x)  
  
plot_value = y_hat.detach().cpu().numpy()
```

Bad use: detaching an intermediate tensor that the loss still needs to differentiate through.

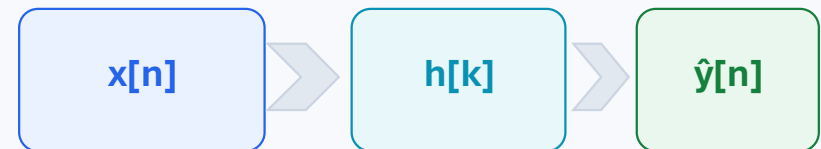
Treat FIR taps as learnable parameters

Now “training” means adjusting filter coefficients to match a desired signal response.

$$\hat{y}[n] = \sum_{k=0}^{M-1} h[k]x[n-k]$$

$$L(\mathbf{h}) = \frac{1}{N} \sum_n (\hat{y}[n] - y_{\text{target}}[n])^2$$

$$\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n]x[n-r]$$



What autograd is doing

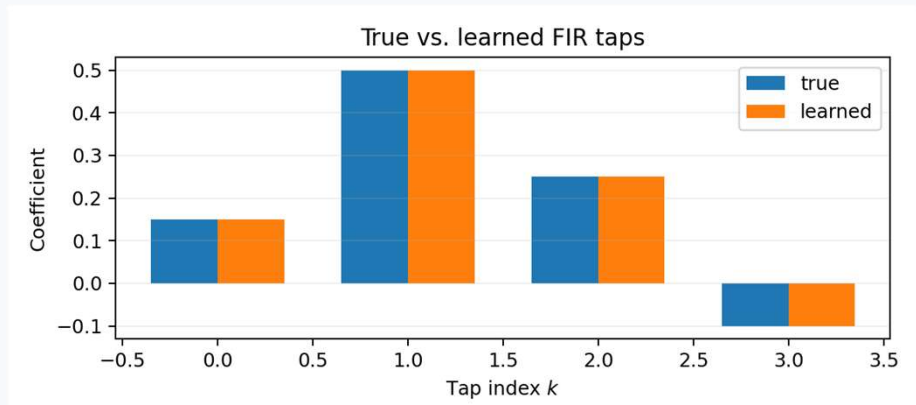
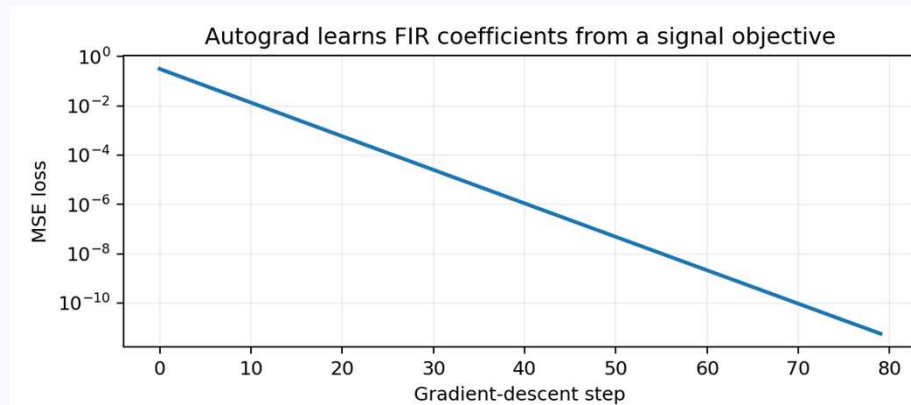
The analytic gradient is correlation-like: error $e[n]$ is correlated with delayed input $x[n-r]$. PyTorch obtains the same result from the graph automatically.

DSP convention note

`torch.nn.functional.conv1d` implements cross-correlation. Flip the kernel if you want the textbook DSP convolution ordering.

Gradient descent recovers the unknown filter

A simple system-identification problem makes autograd tangible.



Target taps

[0.15, 0.50, 0.25, -0.10]

Learned taps

$\approx$ [0.15, 0.50, 0.25, -0.10]

Final MSE

$\approx 5.7 \times 10^{-12}$ in this synthetic example

Interpretation: autograd did not “understand filters.” It differentiated the operations we wrote and gradient descent used those derivatives to update h .

Most early PyTorch mistakes are bookkeeping mistakes

Teach students to diagnose them systematically.

Wrong axis order

(N,L,C) passed to code expecting (N,C,L)

Print shape + name each axis

Mixed device

input on CPU, parameters on accelerator

Move both consistently

Gradient accumulation

forgot `zero_grad()`

Clear intentionally each step

Detached graph

used `detach/no_grad` too early

Preserve required path to loss

In-place surprise

modified a needed tensor in place

Prefer clear functional steps while learning

Lecture 25 synthesis

The conceptual leap is from storing values to storing values plus differentiable history.

DSP arrays $\rightarrow$ tensors $\rightarrow$ computational graph $\rightarrow \nabla_{\theta} L$

Tensors

Same DSP data, explicit shape/dtype/device.

Operations

Broadcasting, matmul, and axis manipulation express vectorized DSP.

Autograd

Forward operations form a dynamic DAG; backward applies the chain rule.

Learning

Unknown DSP parameters can be optimized from $\nabla_{\theta} L$.

Minute paper

In one sentence: what is the difference between “a tensor contains a signal” and “a tensor participates in a computational graph”?

HOMework

Problems 1–3: shapes, broadcasting, and linear transforms

Student version — solutions are in the instructor appendix.

1 • Tensor shapes

State a sensible shape and rank for: (a) one 16,000-sample mono clip, (b) 32 stereo clips, (c) one 257×101 complex STFT, (d) 32 one-channel magnitude spectrograms.

2 • Broadcasting channel gains

x has shape $(24, 2, 8000)$. Apply gain 0.5 to the left channel and 1.2 to the right. Give the gain shape, PyTorch construction, and output shape.

3 • Filterbank matmul

P is 257×100 and W is 40×257 . Write the matrix expression for filterbank energies, give the result shape, and interpret both axes.

HOMEWORK

Problems 4–7: chain rule and graph mechanics

Show your work for the analytic gradients.

4 • Chain rule

Let $u = 2\theta - 3$ and $L = (u + 1)^2$. Derive $dL/d\theta$, evaluate it at $\theta = 4$, and write equivalent autograd code.

5 • Accumulation

For $x = 3$, call backward twice on $L = x^2$ without clearing $x.grad$. What are the gradients after each call, and why?

6 • FIR gradient

For a causal FIR output and MSE loss, derive $\partial L / \partial h[r]$. Interpret the result in DSP terms.

7 • Graph control

Explain `torch.no_grad()` versus `.detach()`. Give one correct use of each and one danger of using either too early.

WEEKLY QUIZ

10 minutes • Questions 1–3

No notes. Show shape reasoning.

Q1

A minibatch contains 16 mono clips, each 4000 samples long. Using the (N,C,L) convention, what is the tensor shape?

Q2

A tensor has shape (8,2,1000). What gain shape applies one independent gain to each channel and broadcasts over batch and time?

Q3

True or false: every PyTorch tensor automatically tracks gradients. Briefly justify.

WEEKLY QUIZ

Questions 4–6

Focus on gradient meaning, not syntax memorization.

$$L = (3\theta + 1)^2, \quad \left. \frac{dL}{d\theta} \right|_{\theta=2} = 42$$

Q4

Compute the derivative shown above.

Q5

Why is reverse-mode autodiff especially appropriate when a model has many parameters but one scalar loss?

Q6

What is the most likely bug if a training loop calls `loss.backward()` every iteration but never clears parameter gradients?

INSTRUCTOR APPENDIX

Homework + Quiz Solutions

Keep these slides after the student-facing material or move them to a separate instructor deck.

SOLUTIONS

quiz
homework

Questions 1–3

A1

(16, 1, 4000). Batch = 16, one mono channel, 4000 time samples.

A2

(1, 2, 1). The length-one axes broadcast over batch and time; the middle axis aligns with channel.

A3

False. Gradient tracking is enabled for appropriate floating/complex tensors when `requires_grad=True`, typically for learnable parameters.

Questions 4–6

$$L = (3\theta + 1)^2, \quad \left. \frac{dL}{d\theta} \right|_{\theta=2} = 42$$

A4

The chain rule gives $2(3\theta+1) \cdot 3$. At $\theta = 2$, the gradient is 42.

A5

Reverse mode reuses one backward traversal to compute the gradient of one scalar objective with respect to many upstream parameters efficiently.

A6

Gradients accumulate across iterations. The optimizer sees old + new gradients unless they are intentionally cleared.

Problems 1–3

Shapes should always be accompanied by axis meaning.

1 • Shapes

(a) (16000,), rank 1. (b) (32,2,16000), rank 3. (c) (257,101), rank 2, complex. (d) often (32,1,257,101), rank 4.

2 • Broadcasting

Use `g = torch.tensor([0.5,1.2]).view(1,2,1)`. Then `y = x*g` has shape (24,2,8000).

3 • Filterbank

$E = WP$, so $(40 \times 257)(257 \times 100) \rightarrow 40 \times 100$. Forty band energies are produced for each of 100 time frames.

Problems 4–5

$$u = 2\theta - 3, \quad L = (u + 1)^2, \quad \frac{dL}{d\theta} = 8\theta - 8$$

4 • Chain rule

At $\theta = 4$, $dL/d\theta = 8(4) - 8 = 24$. Autograd code should set `theta.requires_grad=True`, compute `u` and `L`, then call `L.backward()`.

Solution code

```
theta = torch.tensor(4.0, requires_grad=True)
u = 2*theta - 3
L = (u + 1)**2
L.backward()
print(theta.grad)    # 24
```

5 • Accumulation

For $L=x^2$ at $x=3$, $dL/dx=6$. After the first backward: `grad=6`. After the second without clearing: `grad=12`. `zero_grad()` prevents unintended carry-over.

Teaching point: accumulation is mathematically sensible when multiple graph paths contribute to the same parameter; the bug is accidental accumulation across optimization steps.

Problems 6–7

The FIR derivative is a useful DSP interpretation of backpropagation.

$$\frac{\partial L}{\partial h[r]} = \frac{2}{N} \sum_n e[n] x[n-r]$$

6 • Derivation

With $e[n] = \hat{y}[n] - y[n]$, $dL/dh[r] = (2/N) \sum_n e[n] x[n-r]$. The gradient correlates the current output error with the input delayed to tap r .

7 • no_grad()

Context manager: do not build autograd history for operations inside the block. Example: inference or generating a fixed target.

7 • detach()

Tensor operation: disconnect these values from the existing graph. Example: convert a prediction for plotting without backpropagating through the plotting path.

Danger

Either can silently block gradients if inserted before an operation that the loss still needs to differentiate through.